

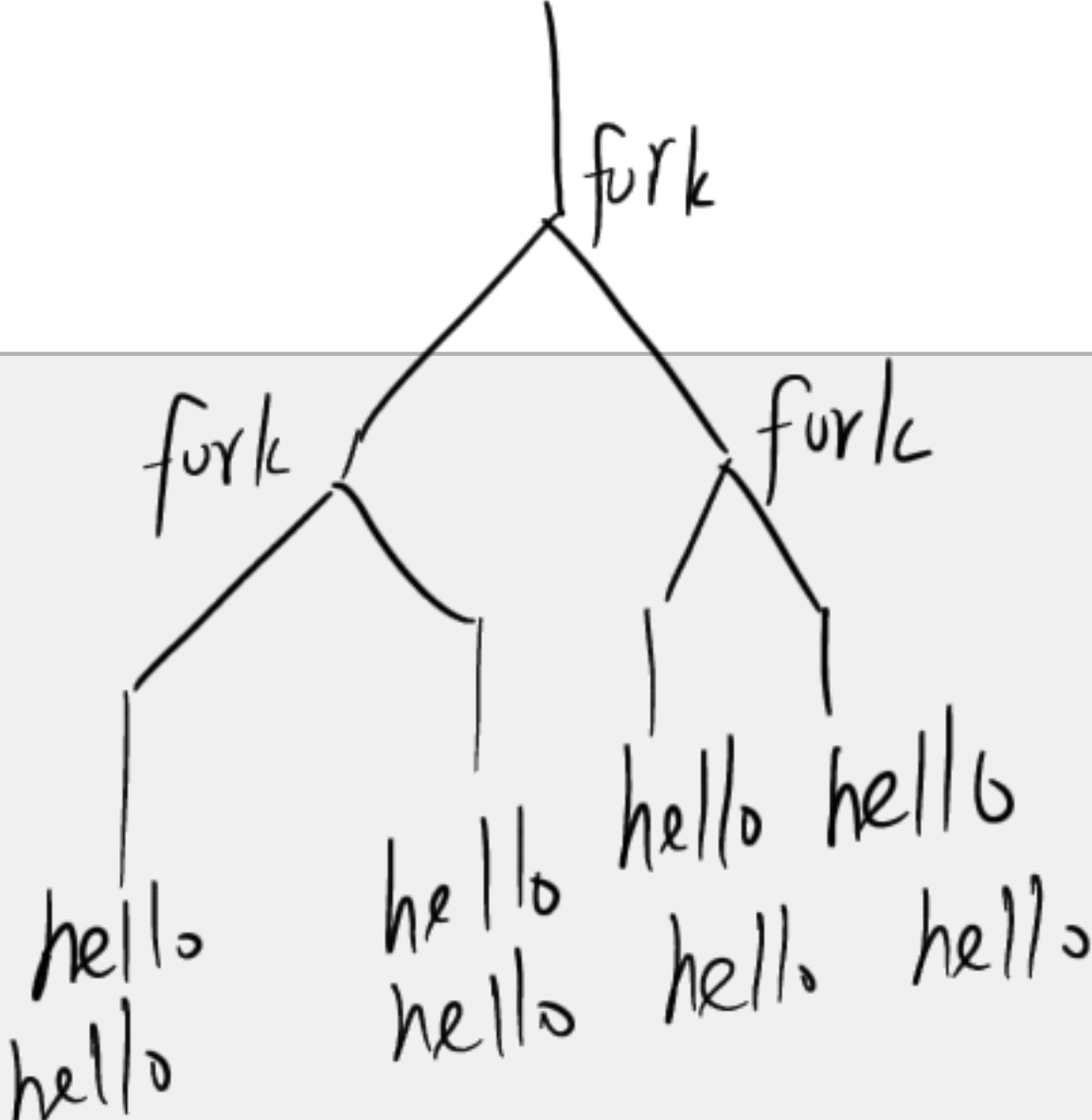
一、Q1 Fork (2分 \* 3)

阅读程序回答问题，并简要说明原因：

1. 该程序共输出多少行 hello? 8

```
void doit() {
    fork();
    fork();
    printf("hello\n");
    return;
}

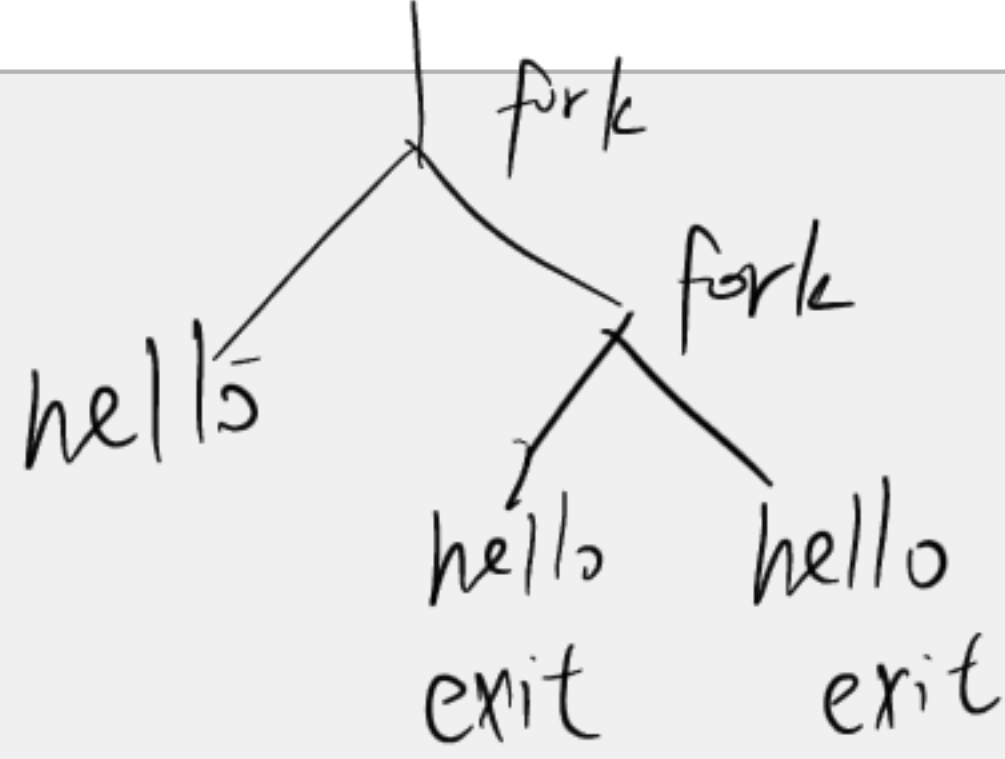
int main() {
    doit();
    printf("hello\n");
    exit(0);
}
```



2. 该程序共输出多少行 hello? 3

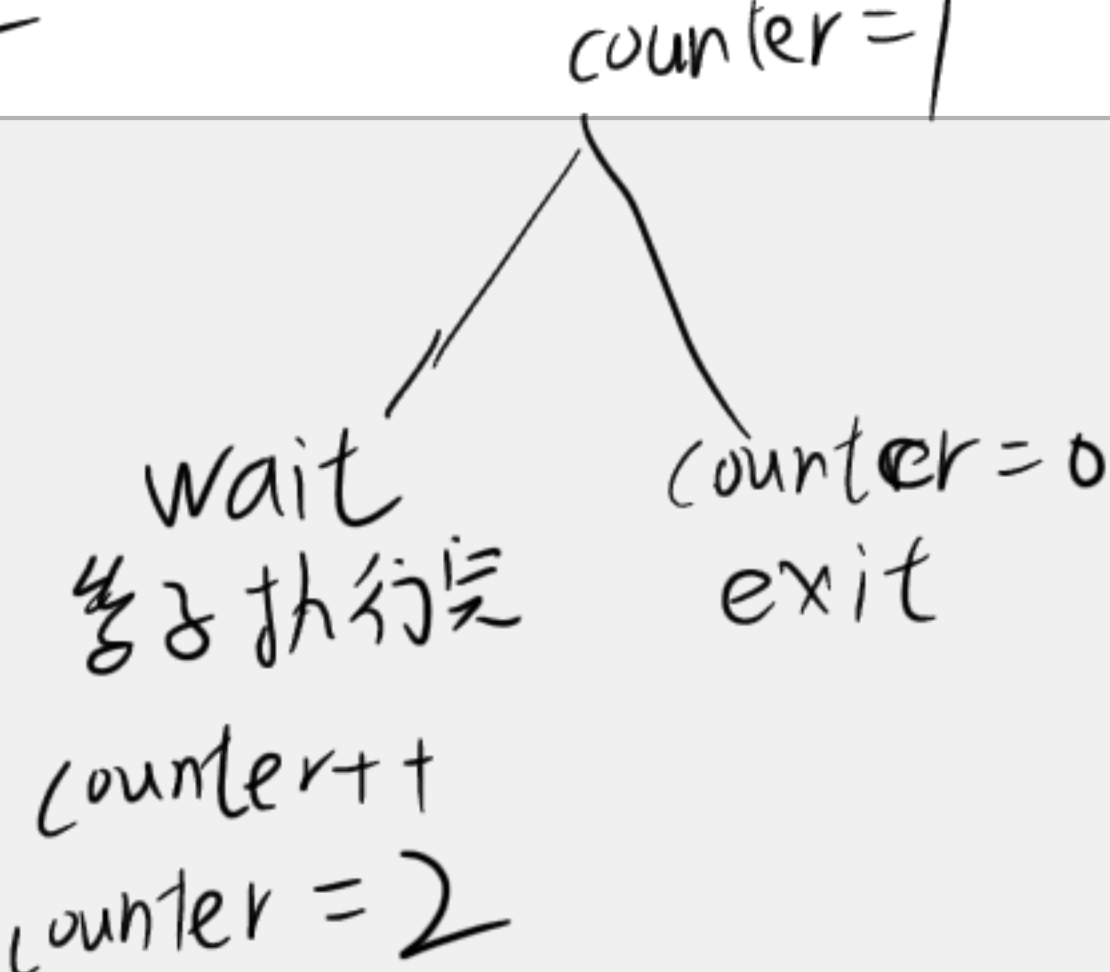
```
void doit() {
    if (fork() == 0) {
        fork();
        printf("hello\n");
        exit(0);
    }
    return;
}

int main() {
    doit();
    printf("hello\n");
    exit(0);
}
```



3. 该程序输出的 counter 值应为多少? 值为2

```
int counter = 1;
int main() {
    if (fork() == 0) {
        counter--;
        exit(0);
    } else {
        wait(NULL);
    }
}
```





```
        counter++;  
        printf("counter = %d\n", counter);  
    }  
    exit(0);  
}
```



## 二、Q2 文件读写 (2 分 + 2 分 \* 4)

假设文件 file1.txt 中有一个字符串 aabbccdd.

下列 C 文件分别被编译成 ./program1 和 ./program2:

```
/* Program 1 */
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main(int argc, char const *argv[]) {
    int pid, fd_x, fd_y, fd_z;
    char buf[8];
```

```
    fd_x = open("file1.txt", O_RDWR);
    fd_y = open("file1.txt", O_RDWR);
    fd_z = open("file1.txt", O_RDWR);
```

```
    read(fd_x, buf, 2);
    read(fd_y, buf + 2, 4);
```

```
// -----
```

```
    if ((pid = fork()) == 0) {
        dup2(fd_x, STDOUT_FILENO);
        dup2(fd_y, STDIN_FILENO);
        execl("program2", "program2", NULL);
    }
```

```
    wait(NULL);
```

```
// -----
```

```
    read(fd_y, buf + 6, 2);
```

```
    write(fd_z, buf + 6, 2);
```

```
    write(fd_x, buf + 4, 2);
```

```
    write(fd_x, buf + 2, 2);
```

```
    close(fd_x);
```

```
    close(fd_y);
```

```
    close(fd_z);
```

```
    return 0;
```

```
}
```

dup2(4,1) : 用 fd4 的内容 替换 fd1  
实现重定向

三者的文件偏移量都为0

buf = [a, a, ?, ?, ?, ?, ?]  
buf = [aa, aa, bb, ?, ?, ?]

子进程 子进程继承父进程的 fd, 共享 offset 和文件内容

用 fd\_x 覆盖标准输出  
用 fd\_y 覆盖标准输入  
执行 program2

此时 fd\_x offset=4  
fd\_y offset=6  
fd\_z offset=0

文件内容 aa cc cc dd

buf = [aa aa bb ??]

buf = [aa aa bb dd]  
buf = [dd cc cc dd]  
buf = [dd cc bb dd]  
buf = [dd cc bb aa]

fd\_y 的 offset=8

fd\_z 的 offset=2

fd\_x 的 offset=6

fd\_x 的 offset=8

从文件读到 buf  
从 buf 写到文件



```

/* Program 2 */
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char const *argv[]) {
    char buf2[2]; 其实是fd-y, offset=4
    read(STDIN_FILENO, buf2, 2); 读入 cc, buf2=[cc], fd-y=offset=6
    write(STDOUT_FILENO, buf2, 2); 写入 bb, 文件内容=[aa cc cc dd]
    }                          fd-x offset=2                      fd-x=offset=4

```

注:

open(filename, O\_RDWR): 打开一个已有文件 filename, 对其进行读写操作, 起始位置为 0;

execl(exename, ...): 执行 exename 程序, 与 execve() 功能类似.

1. 当执行 ./program1 后, 文件 file1.txt 的内容是什么? ddccbbaa

2. 从以下几点解释一下为什么是这个结果 (简述, 言之有理即可):

(1) ./program2 做了什么?

(2) ./program1 在 fork() 之前做了什么, 相应的 buf 的内容如何变化?

(3) ./program1 调用 fork() 了以后, 子进程在干什么?

(4) 子进程返回后, ./program1 又做了什么, 相应的 buf 的内容如何变化?

(1) 它读取标准输入 (即父进程的 fd-y) 的当前位置内容 "cc", 然后写入标准输出 (即父进程的 fd-x) 的当前位置, 将原文件中 "bb" 覆盖为 "cc"

(2) 将 buf 填充为 aaaa bb

(3) 子进程继承父进程的 fd-x 和 fd-y, 共享 offset 和文件内容.

它利用 dup2 重定向流, 实际上推进了 fd-y 的 offset, 并利用 fd-x 修改了文件

(4) 父进程等子进程结束后, 利用已被子进程修改过的 offset 继续操作. 它利用 fd-y 读尾部数据, 利用 fd-x 修改头部, 利用 fd-x 修改中部和尾部, 最终完成数据的顺序拼接



### 三、Q3 信号量（1 分 \* 10）

- 老师的办公室有一个空白板。
- 老师会在白板为空白时往白板上写一道物理题或一道化学题。
- 如果是一道物理题，喜爱物理的小 A 会解答出这道题并把题目擦掉；
- 如果是一道化学题，喜爱化学的小 B 会解答出这道题并把题目擦掉。

请使用信号量和 P、V 原语实现老师、小 A、小 B 三者的同步。

```
sem_t board;      // 白板是否为空
sem_t physics;    // 白板上是否为物理题
sem_t chemistry;  // 白板上是否为化学题

void init() {
    Sem_init(&board, 0, __ (A) 1);
    Sem_init(&physics, 0, __ (B) 0);
    Sem_init(&chemistry, 0, __ (C) 0);
}

void teacher() {
    while (1) {
        Course c = (rand() & 1) ? PHYSICS : CHEMISTRY;
        __ (D) P(&board);
        在白板上写题目;
        if (c == PHYSICS) {
            __ (E) V(&physics);
        } else { // c == CHEMISTRY
            __ (F) V(&chemistry);
        }
    }
}

void studentA() {
    while (1) {
        P(__ (G) &physics);
        解答物理题，将其擦掉;
        V(__ (H) &board);
    }
}

void studentB() {
    while (1) {
        P(__ (I) &chemistry);
        解答化学题，将其擦掉;
        V(__ (J) &board);
    }
}
```



四、Q4 信号处理（3 分）

考虑如下程序：

信号处理函数  
当收到SIGINT时  
打印D并以状态4  
退出

```
void handler (int sig) {  
    printf("D");  
    exit(4);  
}  
  
int main() {  
    int pid, status;  
    signal(SIGINT, handler);  
    printf("A");  
    // -----  
    pid = fork();  
    printf("B");  
    // -----  
    if (pid == 0) {  
        printf("C");  
    } else {  
        kill(pid, SIGINT);  
        waitpid(pid, &status, 0);  
        printf("%d", WEXITSTATUS(status));  
    }  
    printf("E");  
    exit(7);  
}
```

给子进程发信号，尝试杀死子进程  
等待子进程结束，并获取状态，  
打印子进程退出状态码(4/7)

```
graph TD
    A[A] -- fork --> B[B]
    B -- fork --> C[C]
    C --> D[D]
    D --> E[E]
    E --> F[exit(7)]
```

以下哪些是可能的输出结果（多选，所有 printf 都是即时输出）：

(A). ABCBE7E

(B). ABD7E

(C). ABBCE4E

(D). ABCDB4E

(E). ABBD4E

AE  
子快  
父  
子慢  
B C E → exit(7)  
杀子无级  
B → kill (失败) wait (拿到7) → 7 → E  
B (C E) D  
可能在这两个地方停  
父 B → 4 → E



## 五、Q5 线程与子进程 (2 分 \* 4)

阅读程序写结果，要求列出所有可能输出，不考虑进程/线程创建失败的情况，并假设 printf 的输出都是即时的、不会被其他 printf 打断。换行请用 \n 表示。

1.

```
// (1)
#include <pthread.h>
#include <stdio.h>
int a = 0;
void* test(void* ptr) { a++; return NULL; }
int main() {
    pthread_t tid;
    pthread_create(&tid, NULL, test, NULL);
    pthread_join(tid, NULL);
    printf("a=%d\n", a);
    return 0;
}
```

主线程创建子线程 test. 子线程执行 a++  
等待子线程执行完毕  
1\n

2.

```
// (2)
#include <unistd.h>
#include <stdio.h>
int a = 0;
void test() { a++; }
int main() {
    int pid = fork();
    test();
    printf("a=%d\n", a);
    return 0;
}
```

fork  
test() 输出 a  
test() 输出 a  
一旦子进程创建完成，父、子进程就互不相干，变量也相互独立。  
a=1\n a=1\n

3.

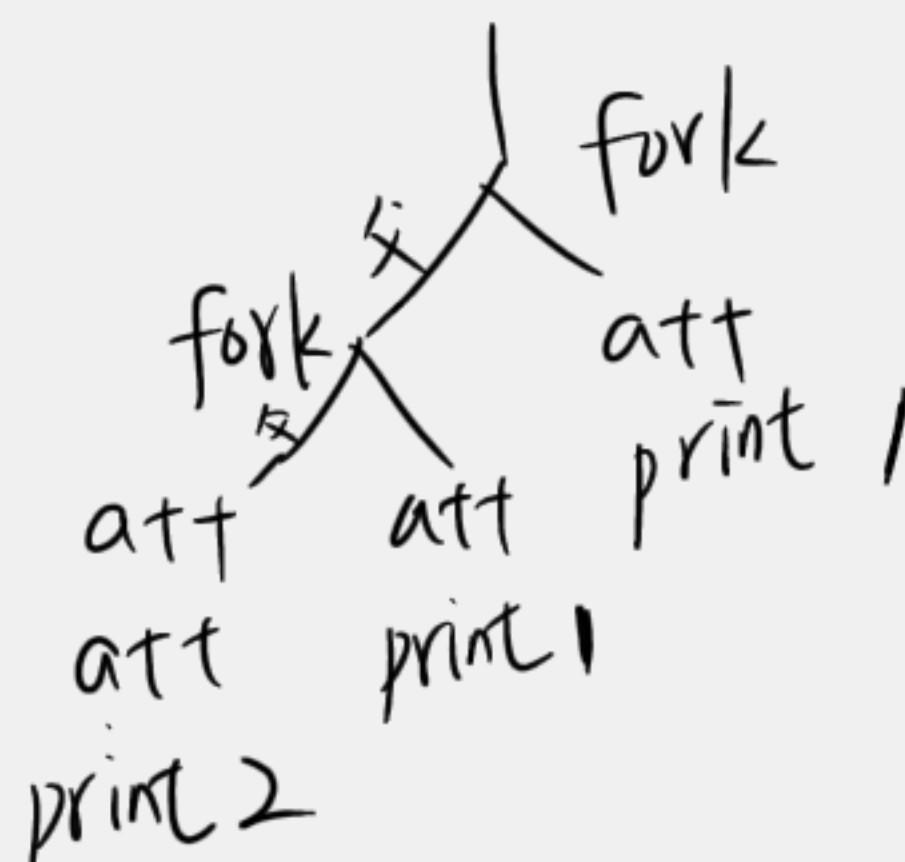
```
// (3)
#include <pthread.h>
#include <stdio.h>
int a = 0;
void* test(void* ptr) { a++; return NULL; }
int main() {
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, test, NULL);
    pthread_create(&tid2, NULL, test, NULL);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    printf("a=%d\n", a);
    return 0;
}
```

等待 tid1 结束  
等待 tid2 结束  
线程1读0写1, 线程2读1写2: a=2\n  
线程1读0, 线程2读0, 写1, 写1. a=1\n



4.

```
// (4)
#include <unistd.h>
#include <stdio.h>
int a = 0;
void test() { a++; }
int main() {
    int pid = fork();
    if (pid != 0) pid = fork();
    test();
    if (pid != 0) test();
    printf("a=%d\n", a);
    return 0;
}
```



$a=1 \backslash n$   $a=1 \backslash n$   $a=2 \backslash n$

或  $a=1 \backslash n$   $a=2 \backslash n$   $a=1 \backslash n$

或  $a=2 \backslash n$   $a=1 \backslash n$   $a=1 \backslash n$

总结: 进程创建完成后, 父进程和子进程就是两个完全不相关的进程了, 变量也是他们各自的, 不共享, 虽然有可能这个执行到一半跑去执行另一个, 再执行一半跑回来继续执行, 但是互不干扰; 但是线程可能互相干扰.